

CROSSING THE EXPERT LINE WITHOUT EXPERT MACHINERY

Yaojian Chen^{1*}, Dazhi Peng^{2*}, Leshan Li¹, Haojun Sun², Yiyun Zheng², Qingfeng He¹,
Zhui Zhu¹, Shengye Xiang¹, Zhen Li², Lin Gan¹, Yuxuan Li^{2†}, Xu Han^{1†}, Zhiyuan Liu^{1†}

¹Tsinghua University ²ModelBest Inc.

ABSTRACT

Fast GPU kernels have always run on expert experience, and the forms that experience takes differ only in how much the human delegates: an expert writes the kernel, or stays in the loop and steers a model through it, or packages what they know into a specialized harness and lets the model search inside it. A specialized kernel agent and a human expert are therefore points on one line. On several complex operators, FlashAttention-4’s backward pass among them, the strongest model beats both the expert kernels and the kernel agents experts built, and it does so in a naive loop, with no specialized harness of its own. It comes out ahead of every specialized agent we could compare against, so the expert experience injected into those agents works against the model. That reopens the question of where human insight should sit in kernel work.

The kernel-specific part of such a harness has three components: evaluation infrastructure, domain knowledge, and workflow. Building them is *loop engineering*, and we do none of it, keeping only the generic agent loop that every coding agent already runs. That is what we call a naive loop: the agent writes its own correctness oracle and its own timing loop. What it has to beat is hand-written expert code, namely FlashAttention-4’s own official kernels and, on the other two operators, whichever of FlashInfer and the winning MLSys-2026 competition kernels is faster, each of them expert-weeks of tuning against this hardware. We measure across three model generations under one stopping rule, on the forward and backward passes of FlashAttention and the forward passes of Gated DeltaNet and DeepSeek Sparse Attention. Opus 4.8 loses to the experts on every operator, at geometric-mean speedups of $0.36\times$ to $0.94\times$. The two generations after it, Opus 5 and Fable 5.1, are at or past the experts on all three, at $1.08\times$ to $1.58\times$, and Fable 5.1 beats FlashAttention-4’s own backward kernels on two of three shapes. The kernel agents fall on the same side of that line: KDA-0.5 reaches $1.45\times$ and $1.28\times$ on the two operators it covers and AKO4X’s archived kernels reach $1.02\times$ and $0.57\times$, where Fable 5.1 reaches $1.58\times$ on both. Kernel work has run this loop before: the model cannot write the kernel, so expertise gets mounted around it, the wrapped system clears the bar, and then a later model does it alone and the mounting comes off. For attention kernels on Blackwell the model has reached that point, beating what experts wrote without a harness that experts tuned for it. Loop engineering therefore has somewhere else to be. What a naive loop already beats does not need it, while the harder operator classes, and the last margin between a fast kernel and the ceiling of the hardware, still do.

1 INTRODUCTION

Fast GPU kernels have always been built out of expert experience, and the forms that experience takes differ only in how much the human hands over. The expert writes the kernel. The expert stays in the loop and steers a model. The expert packages what they know into a harness and lets the model search inside it. Each step delegates more, and the experience stays in the system either way.

*Equal contribution.

†Corresponding authors: Yuxuan Li, Xu Han, Zhiyuan Liu.

Specialized agents for GPU kernel optimization have become a research area of their own along that line. Systems such as Kernel Design Agents (KDA), Agentic Variation Operators (AVO), Agentic Kernel Optimization (AKO), and Compiler-Agent co-design for frontier Kernel Evolution (CAKE) (MIT HAN Lab, 2026; NVIDIA, 2026; AKO Team, 2026a; Ye et al., 2026) wrap a language model in domain machinery, and they wrap it in different things. KDA mounts a curated optimization corpus and a profiler reference as separate skills. AVO keeps a knowledge base and a candidate lineage. CAKE builds a typed IR with a verifier and cost model. AKO reports evaluation, benchmarking, and profiling interfaces. Each of them also prescribes some procedure for searching the implementation space. Over the same period, general-purpose coding agents have acquired *goal modes* that accept an objective, a workspace, and a verifier, and iterate autonomously (Anthropic, 2025; OpenAI, 2025). The two lines have never been measured against each other, and the distinction matters because of where the expertise sits: every crossing of the expert line reported so far has been made by a specialized system, with the expertise mounted around the model rather than held by it.

We decompose a specialized harness into three kernel-specific components: *evaluation infrastructure* (compilation, correctness checking, timing), *domain knowledge* (heuristics, optimization checklists, profiling guides), and a *prescriptive workflow*, the fixed search procedure that orchestrates the model. Building these components is *loop engineering*. The generic agent loop is the given. A specialized kernel agent is that loop with kernel-specific components engineered around it. Our baseline does none of it. It removes all three and keeps only the generic loop every coding agent already runs. We call that a *naive loop*. A general-purpose agent receives one workload shape and a four-sentence goal: measure the reference kernel yourself, beat it, and establish your own tolerance against your own fp32 transcription of the operator. We compare the kernels it produces against the expert references and against the kernels KDA-0.5 publishes, on the forward and backward passes of FlashAttention and the forward passes of Gated DeltaNet and DeepSeek Sparse Attention, re-timing every baseline ourselves. Prompt, harness, and stopping rule are held fixed across three successive frontier models, so the model is the only variable along that axis.

Naive-loop kernels cross two lines as the models improve (Figure 1). The first is the expert-written references: FlashAttention-4’s official kernels, and on GDN and DSA whichever of FlashInfer and the winning MLSys-2026 competition kernels is faster. These are the fastest hand-written code anyone has published for this hardware. The naive loop is below them on every operator at Opus 4.8 and at or above them on all three at Fable 5.1. The second is KDA-0.5, which sits above the expert line on both operators it covers and below Fable 5.1 on both. To our knowledge no prior report has an agent beating expert-written kernels with no kernel-specific machinery at all, in any operator family, and the system closest to ours does not test it: KDA’s own ablation weighs its three skills against one another, never against their absence (HAN Lab Kernel Mafia, 2026a). A case study (Section 5) shows where one of the crossings comes from: the winning naive-loop kernel on GDN prefill makes a choice that falls outside the space the specialized workflow searches.

One turn of a cycle. We read the two crossings as a stage rather than a verdict. On a new operator the model first cannot write the kernel; expertise is mounted around it until the wrapped system can; then the capability moves inside, and a later model writes the kernel without the mounting. For attention kernels on Blackwell that last step has happened. The cycle has not ended: the next architecture, the next DSL, and the next class of operator each start the turn over. The open question is therefore not whether loop engineering is obsolete, but where the next turn takes its leverage from, which is a question about what expertise has not yet been internalized. Our reading, argued in Section 6, is that this is a question of decision rights more than one of knowledge. The workflow above the model fixes the architecture of a kernel. The assembler below it fixes the instruction schedule. Both decide using information the model holds.

That question has a natural instrument. A naive loop carries no kernel-specific machinery, so what it fails at is a direct read on what the model still lacks, uncontaminated by a harness that might be covering for it. On the three operators here it fails at nothing, which is why the measurement reduces to a set of speedups. On the next architecture or the next operator class it will fail somewhere, and that is the signal a harness should be built against: not a checklist assembled in advance, but the gap a bare run exposed. The gap may close through the harnesses themselves: specialized agents produce high-quality kernels at volume, a later model trains on them, and the scaffolding is what makes its own removal possible. We do not test that route.

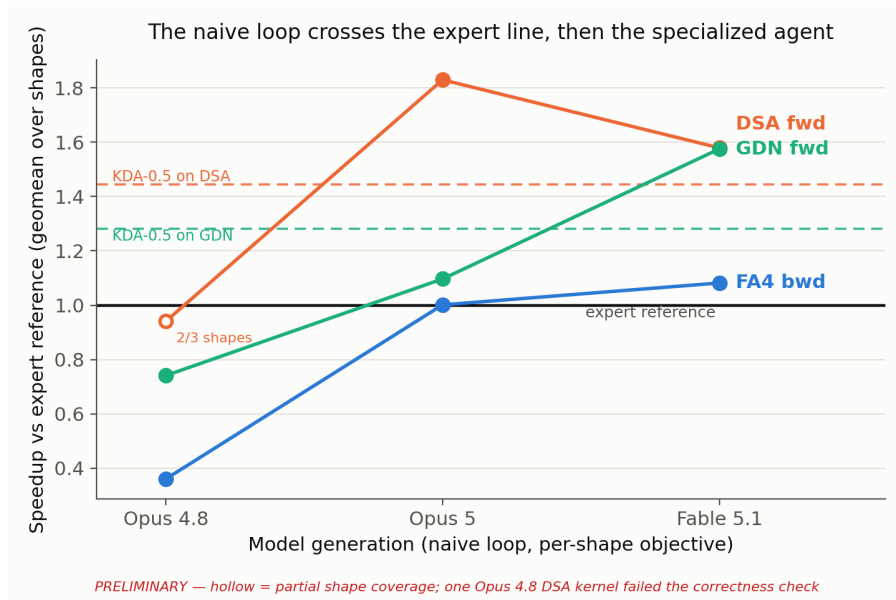


Figure 1: **Crossing the expert line, then the specialized agent.** Geometric-mean speedup of naive-loop kernels over the fastest expert-written reference (bold line at $1.0\times$), computed uniformly as reference latency over measured latency, per operator, across three model generations, with a per-shape objective. Dashed lines mark KDA-0.5’s geomean on the two operators it covers, measured against the same references. Hollow markers cover a subset of shapes (annotated). At Fable 5.1 every operator sits at or above the expert line, and both operators with an agent baseline sit above that agent. Opus 4.8’s DSA mean rests on two of the three shapes: on the third, the kernel it produced did not pass the correctness check, so that cell reports a failure and carries no speedup.

We care about that question beyond kernels. A kernel is a small program with an unforgiving objective: it either produces the right numbers faster than the reference or it does not, and reaching that point demands a model hold a hardware model, a numerical argument, and a search lasting hours in view at once. Few coding tasks score orchestration and code quality that directly, which is what makes kernel optimization worth measuring even for people who will never ship one.

This paper contributes: (1) a measurement of naive-loop kernels against both expert libraries and a specialized kernel agent, across model generations under a common stopping rule, and the two crossings it reveals; (2) a case study of a winning naive-loop kernel whose decisive choice lies outside the template space the specialized workflow searches; and (3) the verification protocol the comparison rests on, run by us rather than by the agents: every kernel we report is scored against a naive fp32 oracle on held-out inputs, at the error scale of the reference it claims to beat.

2 BACKGROUND AND RELATED WORK

2.1 SPECIALIZED KERNEL AGENTS

Specialized kernel agents share a shape: a language model wrapped in domain machinery that supplies what the model is assumed to be missing. We survey the four systems named in Section 1 (MIT HAN Lab, 2026; NVIDIA, 2026; AKO Team, 2026a; Ye et al., 2026) through the three-component lens of Section 1: what evaluation infrastructure each builds, what domain knowledge it encodes, and what workflow it prescribes. Table 1 gives the mapping. The comparison that follows is between whole systems. Two of the four publish both a harness and kernels for the contest shapes we also target. KDA publishes its workflow, its skills, and the per-operator phase prompts of its contest runs (MIT HAN Lab, 2026; HAN Lab Kernel Mafia, 2026a), and KDA-0.5 releases the CuTe-DSL kernels it produced (HAN Lab Kernel Mafia, 2026b). Every KDA kernel we compare against is KDA-0.5’s, released on 2026-08-31, not the team’s original contest submission (KDA

Table 1: Specialized kernel agents decomposed into the three harness components, plus the properties that decide how each system can enter our experiments. KDA’s and AKO’s rows are read off their repositories (MIT HAN Lab, 2026; AKO Team, 2026b;c); AVO’s and CAKE’s are read off their reports (NVIDIA, 2026; Ye et al., 2026), since neither publishes its harness. Where a property is not stated, we say so rather than infer it.

System	Evaluation	Knowledge	Workflow	Scope	Model in reported runs
<i>Publishes harness and kernels; kernels re-measured here</i>					
KDA	Left to the host task; NCU profiler skill	KernelWiki corpus and a Blackwell programming reference, mounted as skills	Task-contract template and a fixed step order; no kernel-specific search strategy	CUDA kernels; B200/B300	Claude, via Claude Code; version unstated
<i>Publishes harness and kernels; slower than KDA-0.5 on our workloads</i>					
AKO	flashinfer-bench (AKO4X) or KernelBench (AKO4ALL); per-operator reference workspaces	AKO4X ships per-DSL skills for six kernel languages, copied into each spawned workspace; AKO4ALL ships an empty hints file instead	Skill protocol: plan before running, iteration loop, stall handling, stopping rule; AKO4X adds multi-round campaigns with cross-run memory and master/sub agents	10 operator families (AKO4X); any kernel (AKO4ALL)	Claude, via Claude Code; Opus 4.7 in AKO4X’s defaults
<i>Surveyed only (harness not released)</i>					
AVO	Execution feedback against cuDNN and FA4	Domain knowledge base; candidate lineage	Evolutionary search with the agent as variation operator	Attention (MHA, GQA)	Internal coding agent; LLM unnamed
CAKE	Typed IR with verifier, cost model, localized diagnostics	Verifier rules and tactics the harness accumulates	Single-shape evolution, then library generalization	Flash-KMeans, KNN, KMeans, linear attention	GPT-5.6-sol at xhigh reasoning

0.1), which is the version that earlier published comparisons measure (AKO Team, 2026c). AKO ships two runnable harnesses, AKO4ALL as a Claude Code skill and AKO4X as a campaign-based multi-agent system over flashinfer-bench, and AKO4X archives the kernels its campaigns produced for each operator, including all six of our DSA and GDN workloads (AKO Team, 2026b;c). We compare against KDA-0.5, the faster of the two on those workloads, and ran neither harness, which leaves a same-model comparison open rather than impossible (Section 6). AVO releases neither an implementation nor kernels. CAKE does release its kernels: the recurrent Kimi Delta Attention prefill and decode kernels it generated are merged into FlashInfer and tracked publicly (FlashInfer contributors, 2026), but that operator is outside the three we target, and the harness that produced them is not released. The rows for AVO and CAKE in Table 1 therefore describe what their reports say about their harnesses.

Why only one baseline. The field is far larger than these four: a survey covers the space (Yu et al., 2026), and a continuously updated index lists around thirty systems, of which roughly a third release code (FlagOS-AI, 2026). Three properties decide which of them can enter a comparison with ours, and most fail on one of them. The first is a released harness rather than a report, which is where AVO and CAKE fail it, and so do most systems on the list. The second is our hardware and our operator family. We target Blackwell because it is NVIDIA’s current flagship data-center generation, the hardware on which frontier models are now trained and served, so a crossing there is one where kernel performance matters most. Several open harnesses are real systems aimed elsewhere: CudaForge pairs a coder agent with a judge agent reading Nsight Compute metrics, and reports on KernelBench across A100- and RTX-class GPUs (Zhang et al., 2025); GEAK targets Triton on AMD MI300X and MI250 (Wang et al., 2025). K-Search maintains a tree of bottleneck hypotheses as a co-evolving world model and releases both its harness and its kernels, but its attention results (GQA and MLA paged decode and prefill) are on H100, and its one Blackwell result is an FP8 MoE kernel (Cao et al., 2026). None of these bars is ours, and speedups quoted across papers are not comparable when the reference, the shape, and the machine all differ. The third is that the harness be separable from the model, which the next paragraph takes up.

Besides KDA, two systems clear all three. AKO runs on B200 over flashinfer-bench and publishes both of its harnesses and its kernels (AKO Team, 2026b;c); it is a measurement we have not made. AdaExplore searches on B200 over production shapes with expert kernels as the baseline where one exists, and releases its harness (Du et al., 2026); its reported crossings are on memory-bound operators rather than the class we target (Section 2.3). K-Search is built on flashinfer-bench as well, so our operators expressed as traces would bring it into the comparison once pointed at our Blackwell operators (Section 6). Both KDA-0.5 and AKO4X publish kernels for the six DSA and GDN workloads we target. Re-timed on our node, AKO4X’s are slower than KDA-0.5’s on all six (Table 3), which is why KDA-0.5 sets the specialized-agent bar here. AKO reports itself ahead of KDA on every contest kernel, at $31.1\times$ against $4.6\times$ on DSA and $2.37\times$ against $1.72\times$ on GDN prefill (AKO Team, 2026a). That head-to-head is against KDA 0.1, the team’s original contest submission. KDA-0.5, the later release (2026-08-31), is the version that passes AKO4X, and it is the one we measure.

A separate line trains the model instead of wrapping it. Kevin-32B applies multi-turn RL to CUDA generation and lifts its base model from $0.53\times$ to $1.10\times$ over PyTorch eager (Cognition, 2025), and CUDA Agent scales agentic RL inside a verification-and-profiling environment, reporting KernelBench results above frontier proprietary models (Dai et al., 2026). We do not compare against that line either, for two reasons. Its numbers are measured against an eager-mode baseline on KernelBench-style tasks, so $1.10\times$ there and $1.58\times$ over FlashInfer here are distances from different bars. And training the model is a different intervention from changing the harness: CUDA Agent ships its trained model together with its environment, so a comparison against it varies both at once and could not attribute the difference to either.

These systems continue a long tradition. Kernel autotuners and search-based compilers (Chen et al., 2018; Zheng et al., 2020; Tillet et al., 2019) and superoptimization (Schkufza et al., 2013) encode expert knowledge and a search procedure into infrastructure; specialized agents are the same impulse applied to language models. That history is relevant here because it has played out before: hand-engineered components have repeatedly given way to learned ones as the underlying models improved (Sutton, 2019).

2.2 GENERIC AGENT LOOPS AND LOOP ENGINEERING

General-purpose coding agents such as Claude Code and Codex (Anthropic, 2025; OpenAI, 2025) have acquired *goal modes*: the agent receives an objective, a workspace, and a verifier, and iterates autonomously, editing, running, and measuring without a prescribed procedure. Such a mode is not the absence of a harness; it is a generic agentic loop, and it is the same substrate the specialized agents of Section 2.1 build on: KDA and AKO both run on Claude Code. We therefore separate two layers. The *generic harness* is the agent loop itself, with its tool use, context management, and autonomous continuation. The *kernel-specific harness* is what loop engineering adds on top. It is what the three components of Section 1 describe. A *naive loop* is the first layer with none of the second. That is what Section 3 runs. Outside kernel optimization, several studies report that the gains from elaborate agent scaffolds shrink as the underlying model improves (Xia et al., 2024; Kapoor et al., 2024); we measure this effect in the kernel domain by holding a naive loop fixed across three model generations.

2.3 KERNEL BENCHMARKS AND NUMERICAL-TOLERANCE PITFALLS

KernelBench and successor evaluations (Ouyang et al., 2025; Wijk et al., 2024; AgentKernelArena authors, 2026) established standardized tasks for LLM kernel generation, and also surfaced a recurring failure mode: kernels that pass fixed atol/rtol checks while silently degrading numerical quality, trading accuracy for speed up to whatever the tolerance permits. In one prominent episode, reported speedups above $100\times$ were traced to kernels exploiting weaknesses of the evaluation harness itself (Sakana AI, 2025). Because our agents optimize against the evaluation in their loop, this failure mode is a first-order threat to any speed comparison. Our response has two parts: the goal itself tells the agent to build its bound from the reference kernel’s own error against an fp32 oracle rather than from a fixed tolerance (Appendix B), and every number we report is re-checked afterwards on held-out inputs the agent never observes (Section 3.3). Benchmarks also evaluate a kernel across a fixed

task list; production serving instead poses many shapes per operator, which is why our optimization objectives are per-shape (Section 3.2).

Which line is being crossed. Speedups in this literature are quoted against different denominators, and the gap between them is wide. KernelBench and the systems evaluated on it measure against Torch Eager: `fast_p` is defined as a runtime ratio over the eager implementation, so clearing that line means producing a usable kernel. Our denominator is the fastest expert-written implementation of each operator, where the remaining headroom is smallest, and the operators differ in how small. AdaExplore, searching on B200 over production shapes with expert baselines available, passes expert-written CUDA on memory-bound operators such as RMSNorm, and names compute-intensive kernels carrying architecture-specific instructions as the class that stays out of reach because human optimization has already been heavy there (Du et al., 2026). The three operators in this paper are in that second class: each reference is hand-tuned for the target architecture, and two of the three were written for a competition whose object was to make them as fast as the hardware allows.

Positioning. Prior comparisons of kernel agents hold the harness fixed and vary the model, or vary both at once; none isolates the harness itself. We hold a naive loop fixed across model generations and measure what it produces against both expert libraries and a specialized agent’s own kernels.

3 EXPERIMENTAL SETUP

3.1 THE NAIVE LOOP

The naive loop receives two things: a target operator pinned to one concrete shape configuration, and a four-sentence goal. The goal says to measure the reference kernel’s device time on the target hardware, then write the operator from scratch and beat that number, with no constraint on how: no required algorithm, no template to fill, no prescribed sequence of optimization steps. Its last two sentences say how to stay honest about accuracy: write a naive fp32 transcription of the operator, score the reference against it, and use the reference’s own error as the tolerance the new kernel has to meet. The oracle, the comparison, and the timing loop are all code the agent writes. The prompt is reproduced verbatim in Appendix B.

Relative to a specialized harness, this removes all three kernel-specific components. Removed with the knowledge component: curated optimization checklists, hardware-specific heuristics, profiler integration, and any skill that interprets profiler output for the model. Removed with the workflow component: multi-stage search procedures, template libraries, and any orchestration that decides what the model attempts next. Removed with the evaluation component: the fp32 oracle, the correctness gate, and the device-time measurement, which a specialized harness ships as infrastructure the agent calls and which here are the agent’s own first task. What remains is the generic harness of Section 2.2, which any coding agent already has, plus the goal naming the target. The agent may still profile, if it decides to and writes the invocation itself; what it does not get is a harness component that profiles on its behalf.

3.2 WORKLOADS AND SHAPES

We evaluate three operators: FlashAttention (FA) (Dao et al., 2022; Shah et al., 2024; Dao et al., 2026), in both its forward and backward passes, and the forward passes of Gated DeltaNet prefill (GDN) (Yang et al., 2025) and DeepSeek Sparse Attention (DSA) (DeepSeek-AI, 2025b). **[TODO: Many more operators will be added; update this list, the three selection axes below, and the attention-family scope statement accordingly.]** The selection is deliberate on three axes. First, the operators span maturity: FA has been public since 2022 and is represented in every serving stack and, presumably, every training corpus; GDN was published in early 2025 and ships in production: Qwen3-Next interleaves it with gated attention at a 3:1 layer ratio (Qwen Team, 2025); DSA appeared in late 2025. That spread lets Section 4 ask whether the crossing point tracks how much operator-specific knowledge a model has internalized. Second, every operator has a strong expert-written reference, and where more than one exists we measure against the fastest. For FA that is the official kernels. For GDN and DSA we put FlashInfer (Ye et al., 2025) against the winning MLSys-2026 competition

kernels and take whichever wins each shape; both beat the Flash Linear Attention library (Yang & Zhang, 2024), and on DSA both beat DeepSeek’s own FlashMLA kernels (DeepSeek-AI, 2025a), so neither of those sets the bar. Neither surviving source wins uniformly, FlashInfer taking three of the six shapes and the competition kernels the other three (Table 3), which makes the composite $1.0\times$ line harder to clear than either source alone. Third, all three sit squarely inside the specialized agents’ advertised domain, so the baselines compete on home turf. The deliberate cost of this selection is scope: all three are attention-family operators, and we state our conclusions for that family (Section 6). The limit is on what we measured rather than on what the setup can address. A four-sentence goal naming an operator and a shape carries no operator knowledge and no attention-specific assumption, so widening the sweep is a matter of running more operators, not of building anything further; we chose these three because their references are the hardest to pass, not because they are the ones the method fits. All measurements are taken on a single NVIDIA B200 (compute capability 10.0, 183 GB HBM) with driver 580.105.08 (open kernel module) on host kernel 5.15.0-126, which caps usable CUDA at 13.0. Inside the container (NGC PyTorch 25.04 on Ubuntu 24.04.2) we build with CUDA 12.9.41 and GCC 13.3.0 against PyTorch 2.7.0a0, Triton 3.2.0, and cuDNN 9.9.0; Nsight Compute 2025.2.0 is available should an agent choose to profile. The DSA and GDN shapes are MLSys-2026 competition workloads and the FA shapes are standard training-scale configurations; Appendix A lists every one in full.

3.3 AGENTS, MODELS, AND BUDGET ACCOUNTING

The naive loop is run with three successive frontier models (Opus 4.8, Opus 5, and Fable 5.1), holding the harness, the prompt, and the shapes fixed, so that the model is the only variable along that axis.

Which specialized baselines can be compared, and how, is constrained by what we can measure. KDA-0.5¹ publishes its CuTe-DSL kernels, which we re-time under our own setup rather than quoting its reported figures; it is the specialized baseline in Table 3. AKO publishes two runnable harnesses, and AKO4X’s repository archives kernels for all six of our DSA and GDN workloads (AKO Team, 2026b;c); we re-time those the same way, and they are slower than KDA-0.5’s on each (Section 2.1), so KDA-0.5 alone sets the specialized-agent bar. We have not run either system’s harness with our own models. AVO releases neither an implementation nor kernels, so it cannot enter a comparison at all. CAKE’s kernels are public (FlashInfer contributors, 2026) but cover Kimi Delta Attention, which none of our shapes exercises, and its harness is not released, so it enters neither a kernel comparison nor a same-model one here.

Validating what we report. The tolerance the agent derives is part of the object under study, so it is not what our numbers rest on. Every kernel in Table 3, ours and the baselines alike, is re-timed and re-checked by us: correctness is scored against a naive fp32 implementation of the operator (a direct transcription of the mathematical definition, with no blocking, no fusion, and no reduced-precision accumulation) on held-out inputs the agent never saw, at the error scale of the reference kernel being compared against, recomputed per shape and per output tensor. A kernel we report as faster than its reference is therefore never also less accurate than it, and no speedup here is bought with precision. This also separates kernels that are correct from kernels fitted to the inputs they were shown. It is measurement of our claims, not a component of the agent’s harness.

Budgets. Every naive-loop run follows the same stopping rule: the agent runs until twenty consecutive evaluation rounds yield no improvement. No wall-clock, token, or evaluation-count ceiling is imposed on top of it. Table 2 reports the point at which each run’s last improvement landed rather than its total wall-clock time, because the twenty idle rounds that follow are an artifact of the stopping rule and not work the model needed; on that measure a kernel reaches its final form in 1.5 to 37.2 hours. A common stopping rule is what makes the comparison across models fair: every cell is a converged result under an identical criterion, so the spread reflects how long each model needed rather than where we chose to cut it off. Against the specialized baselines the comparison

¹A pre-1.0 release, covering the DSA and GDN operators only: the CuTe-DSL kernels at contest-solution commit 7aad0429 (2026-08-31) (HAN Lab Kernel Mafia, 2026b). It is distinct from KDA 0.1, the team’s original contest submission; every KDA number in this paper, including Table 3, Figure 1, and Section 5, is KDA-0.5.

Table 2: Agent time per shape, in hours to the last improvement rather than total wall-clock time (Section 3.3). The *fail* cell is the run of Table 3 that did not pass the correctness check, which leaves it no last improvement to time. Dashes carry the same meaning as in Table 3.

Operator	Shape	Opus 4.8	Opus 5	Fable 5.1
FA4 bwd	MHA	12	18.25	8.2
	GQA	12	37.2	3.75
	varlen MHA	12	22	8.2
DSA fwd	heavy load	<i>fail</i>	1.5	5.9
	small batch	24	4	5
	mixed lengths	7.2	8.7	1.5
GDN fwd	long sequence	9	5.8	8.5
	single short seq.	13.1	8.5	7.5
	few long seqs.	10.3	5.8	7.5
MLA decode	single mid-length	—	—	0.5
	mid batch, mixed	—	—	1
	large batch	—	—	0.8
MLA prefill	short prefill	—	1.9	—
	mid prefill	—	1	—
	mixed large prefill	—	3.3	—
GQA decode	single request	—	1.5	—
	mid batch, short seqs.	—	—	1.4
	large batch	—	0.6	—

is weaker, since their search procedures define their own stopping conditions and cannot be placed under ours (Section 6). We report agent time and not token cost; the two are not interchangeable, and we measure only the former.

4 CROSSING THE EXPERT LINE, THEN THE AGENT

Table 3 reports the per-shape speedups; Figure 1 aggregates them per operator, and Table 5 gives the latencies behind them. Four observations.

First, the trend across model generations is steep. Opus 4.8 sits below the expert line on every operator, at geomeans of $0.36\times$ to $0.94\times$. Its DSA mean covers two of the three shapes; on the third, the kernel that run produced did not pass our correctness check, so the cell reports a failure instead of a speedup. Opus 5 is at or above it on all three ($1.00\times$ on FA4 backward, $1.83\times$ on DSA, $1.10\times$ on GDN); Fable 5.1 widens the margin on the two where the line was only just cleared, reaching $1.08\times$ on FA4 backward, $1.58\times$ on DSA, and $1.58\times$ on GDN prefill.

Second, the naive loop passes the specialized agent as well, and this is the comparison the specialized-agent question turns on. On the two operators KDA-0.5 covers, its geomean speedups over the same references are $1.45\times$ on DSA and $1.28\times$ on GDN; Fable 5.1 reaches $1.58\times$ on both. Opus 5 is already past KDA-0.5 on DSA ($1.83\times$) but still below it on GDN ($1.10\times$), which places the second crossing one generation after the first. Table 4 gives the head-to-head ratios per shape, and there the picture is less uniform than the per-operator means: Fable 5.1 wins five of the six shapes where both systems run, taking all three on GDN and losing only DSA heavy load, where KDA-0.5’s kernel is 6% faster. AKO4X’s archived kernels are the weaker of the two specialized baselines on our shapes: $1.02\times$ on DSA and $0.57\times$ on GDN, with all three GDN shapes below the expert line (Section 2.1).

Third, the crossing does not track operator recency. FA, the operator best represented in public code, is crossed last and by the narrowest margin, and on FA4 MHA the best naive-loop kernel is still 5% slower than the official one. The references differ in the same direction: FA’s is the official kernel, whose published lineage runs from 2022 to 2026 (Dao et al., 2022; Shah et al., 2024; Dao et al., 2026), while the GDN and DSA bars come from a library and a competition on operators first published in 2025 (Section 3.2). What the naive loop has to clear is the reference, and how much of an operator a model has seen does not predict how hard that is.

Table 3: Speedup over the expert reference, per shape (preliminary). Every speedup is computed uniformly as reference latency over measured latency. The reference is the fastest expert kernel we measured for that shape: on GDN and DSA, whichever of FlashInfer and the winning MLSys-2026 competition kernel (Dogacel on DSA, Kachua on GDN) is faster, named in the Source column. DeepSeek’s own FlashMLA kernels were measured on DSA and are slower than FlashInfer on every shape, so they never set the bar. The same kernels measured against the specialized agents are in Table 4; absolute latencies are in Table 5 and agent time in Table 2. A cell marked *fail* is a run whose kernel did not pass the correctness protocol of Section 3.3; it has no speedup to report and is excluded from every mean. **A red dash marks a run we have not made: the paged operators were run on the two later models only.**

Operator	Shape	Ref (μ s)	Source	Opus 4.8	Opus 5	Fable 5.1
FA4 bwd	MHA	1387	FA4 official	0.22×	1.01×	0.95×
	GQA	1459	FA4 official	0.40×	1.11×	1.24×
	varlen MHA	1639	FA4 official	0.54×	0.89×	1.08×
DSA fwd	heavy load	13	MLSys’26	<i>fail</i>	1.60×	1.18×
	small batch	5	MLSys’26	1.58×	1.77×	1.56×
	mixed lengths	11.07	FlashInfer	0.56×	2.16×	2.13×
GDN fwd	long sequence	145.6	FlashInfer	0.94×	0.81×	1.54×
	single short seq.	7	MLSys’26	0.76×	1.63×	1.81×
	few long seqs.	133	FlashInfer	0.57×	1.00×	1.40×
MLA decode	single mid-length	15.98	FlashInfer	—	—	2.47×
	mid batch, mixed	31.25	FlashInfer	—	—	2.37×
	large batch	77.5	FlashInfer	—	—	1.66×
MLA prefill	short prefill	22.38	FlashInfer	—	5.69×	—
	mid prefill	26.46	FA4 4.0.0b19	—	2.03×	—
	mixed large prefill	709.2	FA4 4.0.0b32	—	1.43×	—
GQA decode	single request	12.07	FlashInfer	—	1.52×	—
	mid batch, short seqs.	7.26	FA4 official	—	—	1.83×
	large batch	74.8	FA4 official	—	1.51×	—

Table 4: The same naive-loop kernels measured against each specialized kernel agent, per shape. Each cell is the agent’s latency over ours, so a value above 1.0×

Operator	Shape	vs KDA-0.5			vs AKO4X		
		Opus 4.8	Opus 5	Fable 5.1	Opus 4.8	Opus 5	Fable 5.1
DSA fwd	heavy load	<i>fail</i>	1.27×	0.94×	<i>fail</i>	1.40×	1.04×
	small batch	1.07×	1.20×	1.06×	2.27×	2.55×	2.25×
	mixed lengths	0.34×	1.33×	1.31×	0.42×	1.60×	1.58×
GDN fwd	long sequence	0.81×	0.70×	1.33×	1.42×	1.23×	2.34×
	single short seq.	0.53×	1.14×	1.27×	1.92×	4.10×	4.56×
	few long seqs.	0.45×	0.79×	1.11×	0.83×	1.45×	2.03×

Fourth, these kernels cost hours of agent time per shape: the last improvement lands between 1.5 and 37.2 hours into a run (Table 2). That cost is paid once per shape and amortized over serving.

5 CASE STUDY: WHERE THE NAIVE LOOP WINS

On GDN prefill forward, the Fable 5.1 naive-loop kernel reaches 94.5 μ s on the long-sequence shape, 1.54×

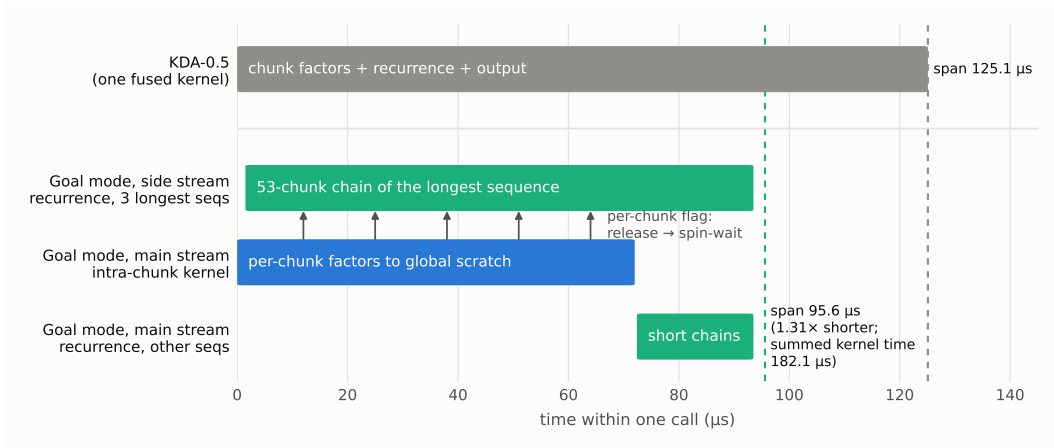


Figure 2: **GDN prefill, long-sequence shape: one call, two organizations.** KDA-0.5 runs the operator as one fused kernel ($125.1 \mu s$). The naive-loop kernel splits it into an intra-chunk kernel and a recurrence kernel, launches the recurrence for the three longest sequences first on a highest-priority side stream, and hands each chunk over through a global flag; its span is $95.6 \mu s$ although its summed kernel time is $182.1 \mu s$. **DRAFT, schematic: the intra-kernel end ($\approx 72 \mu s$) and the flag positions are approximate; a to-scale version needs the full per-item timestamps of the instrumented run.**

The two kernels agree on nearly every local decision. Both factor the recurrence the same way: 64-token chunks, a WY-form system per chunk solved by a hierarchical blockwise inverse, and a chunked delta rule over the resulting factors. Both are warp-specialized persistent kernels built on `tcgen05`, keeping the 128×128 recurrent state in tensor memory and feeding the MMAs through TMA. The naive-loop agent reached that vocabulary without reading any of it. It spent its first four hours on an `mma.sync` implementation that was correct and slow ($0.79\times$), measured that instruction’s 558 TFLOPS ceiling on the machine itself, and only then rewrote the recurrence around `tcgen05`, recovering the descriptor encodings from public PTX and CUTLASS documentation.

They differ in how many kernels run at once. KDA-0.5 emits one kernel: a single device entry point, launched once per call on the current stream, holding the state in tensor memory from the first chunk to the last and never writing a chunk-local intermediate to global memory (HAN Lab Kernel Mafia, 2026b). The naive-loop implementation splits the operator in two — an intra-chunk kernel that computes the per-chunk factors and stores them to global scratch, and a recurrence kernel that consumes them in order. Run back to back the two halves are *slower* than the fused reference, 125 to $134 \mu s$ across the run’s late versions, because the split pays a global round trip the fused kernel avoids. What more than recovers that loss is running the two concurrently. The recurrence kernel for the three longest sequences is launched first on a separate highest-priority stream, so its CTAs take SMs before the intra-chunk kernel’s; each intra-chunk CTA then publishes its chunk with a release store to a global flag, and the recurrence kernel spin-waits per chunk on that flag. An instrumented run shows the longest sequence’s 53-chunk chain occupying the kernel from 1.5 to $93.5 \mu s$ while the nineteen shorter sequences’ recurrences pack into its final twenty microseconds. The result is a kernel span of $95.6 \mu s$ against $182.1 \mu s$ of summed kernel execution: the implementation does $1.46\times$ the reference’s total GPU work and still finishes in three quarters of the time. Device time, as the competition harness and we both define it, is a span.

The specialized system never uses that dimension. KDA’s competition workflow runs in three stages — retrieve production kernel knowledge, iterate on one candidate under profiler evidence, then autotune into a shape-aware dispatcher (HAN Lab Kernel Mafia, 2026a) — and none of the three has concurrency as its axis. The GDN prefill phase prompt that drives the middle stage enumerates the directions to explore: chunking the recurrence, parallelizing within and across sequences, short-sequence fast paths, long-sequence persistent kernels, state layout, vectorized loads, gate precomputation, writeback cost, occupancy, register pressure, memory stalls, TMA, TMEM,

tcgen05. Every item is a property of one kernel. The autotune stage then varies *which* kernel serves a shape, not how many run at once. The released artifact matches: across all three of its GDN paths there is no second stream, no stream priority and no cross-kernel flag; every launch goes to the current stream, and the one path that does issue three kernels documents them as stream-ordered, sequential by construction (HAN Lab Kernel Mafia, 2026b). Provenance points the same way — the kernel for this shape derives from FlashInfer’s SM100 GDN implementation, so the search began inside an existing kernel’s architecture and improved it there. KDA-0.5 had in fact diagnosed this shape’s bottleneck: its long-chain path exists to cut a sequential chunk chain into parallel parts, but its dispatcher gates that path to at most five sequences, and the long-sequence shape has twenty. The naive-loop agent attacked the same chain, and overlapped it with every other sequence’s chunk work instead of splitting it.

How far this case goes. The margin came from a choice the prescribed workflow’s three axes do not range over, and that system’s own output leaves the dimension unused across all one hundred competition workloads. So the scaffolding bounded the search. It did not neutrally accelerate it. It is not evidence that the scaffolding forbids the choice: KDA’s report states that its plans may describe any search strategy and that different shape regimes may use structurally different implementations (HAN Lab Kernel Mafia, 2026a), so the same harness could in principle propose two concurrent kernels. We cannot separate a workflow that excludes the choice from one that never proposed it, and a single case study on a single shape says nothing about how often that distinction costs anything. It does show what the risk looks like. A prescribed trajectory is a prior over implementations. A model strong enough to be asked only for latency will sometimes leave that prior.

6 DISCUSSION AND LIMITATIONS

Where loop engineering should go next. Three of our findings point the same way about what it should be spent on. A prescribed trajectory is a prior over implementations, and on GDN prefill that prior excluded the kernel that won. KDA’s own skill ablation puts its planning framework ahead of its kernel corpus (HAN Lab Kernel Mafia, 2026a), so the corpus was not carrying it either. And the one component we could not do without is the one we ran by hand: correctness against an oracle on held-out inputs. So build the parts of a harness that do not get cheaper as the model improves. Hardware access stays expensive. Production shape traces stay expensive. A correctness definition someone owns stays expensive, and so does a timer that cannot be gamed. Optimization checklists and fixed search orders do not. CAKE argues that the environment, a typed IR with a verifier and a cost model, is where harness value lives (Ye et al., 2026); under our decomposition that is a claim about evaluation infrastructure, and it agrees with what we see. If orchestration survives at all, it should propose rather than restrict. A suggestion the model can decline costs nothing. A template space it cannot leave cost KDA-0.5 a factor of $1.33\times$ on the shape in Section 5.

Removing the evaluation component did not mean the kernels went unchecked. It meant the check was the agent’s own code and that we verified the result afterwards (Section 3.3). Whether a self-written check can be trusted without that backstop is a separate question, and these runs cannot answer it. The history is not encouraging: kernels have passed weak checks while trading accuracy for speed, and that failure mode has shipped in public claims (Sakana AI, 2025). One run in our own set failed that way: the Opus 4.8 kernel for the DSA heavy-load shape optimized against its own check and did not survive ours, and Table 3 reports it as a failure. Someone has to own the definition of correct. That is why we expect evaluation to be the durable component, not because a frontier model cannot write the check.

Where the model has to get better. The open question is no longer whether a frontier model can reach an expert kernel. It is what that costs. A naive-loop kernel takes 1.5 to 37.2 hours of agent time per shape before its last improvement lands, and the stopping rule spends twenty idle evaluation rounds after that. Sample efficiency inside this loop is the axis to push. Two further gaps are visible. One is the class of kernel still out of reach. AdaExplore passes expert CUDA on memory-bound operators and names compute-intensive, architecture-specialized kernels as the ones heavy human optimization still holds (Du et al., 2026); attention on Blackwell is now inside the frontier and GEMM-class kernels are not. The other is generality. Our objectives are per-shape, and a library has to be correct and fast across every shape it serves. No model has been shown to write

one kernel that covers twenty shapes as well as twenty kernels cover them. Those three gaps are what loop engineering now has to buy. On attention at these shapes it no longer has to.

What the next turn should hand over. Two layers decide for the model. Above it, a prescriptive workflow fixes the architecture and leaves the model to tune inside it; the agents a recent survey covers are overwhelmingly iterative local refinement rather than architectural exploration (Yu et al., 2026), and K-Search is among the few that promote a structural choice to an explicit action (Cao et al., 2026). Below it, the assembler owns instruction scheduling and register allocation. Neither layer is idle, and neither is doing its job badly. The problem is that both decide using information the model has and they do not. A workflow that enumerates single-kernel directions cannot know the operator admits two concurrent kernels. `ptxas` allocating registers does not know which values a hand-designed pipeline needs resident, and the author can only hint at the budget through `__launch_bounds__`. The run in Section 5 reaches across both ends in one trajectory: the agent measured `mma.sync`’s ceiling on the machine itself, concluded that instruction family could not reach the target, then rewrote the recurrence around `tcgen05`. It reasoned quantitatively at the bottom and restructured at the top. No harness component offered it either one. Hand scheduling assembly has always been a bad trade, because it binds the work to one hardware generation. An agent changes the denominator: expert-weeks become agent-hours. Redoing it per generation turns into an operating cost. Handing either decision down is also loop engineering. Today’s components give the model a list of what to try. This kind would give it a wider set of decisions to make. We tested none of this. It is where we would look next.

Cost, and the human role. A naive-loop kernel costs hours of agent time per shape (Table 2); for a serving fleet this is paid once per workload shape and amortized across every request that hits it. This does not make expert-written libraries obsolete: a library’s job is to be correct and fast across all shapes, hardware revisions, and years of maintenance, and per-shape wins do not replace that. What shifts is the human role: from writing the fastest kernel for each shape toward defining correctness, maintaining references, and deciding where specialized performance is worth buying.

Limitations. Both crossings compare kernels, not harnesses driven by the same model. Running a specialized harness on our own models is the measurement that separates a harness from the model behind it; KDA and AKO publish harnesses that would allow it (MIT HAN Lab, 2026; AKO Team, 2026b;c), and we ran neither. So we cannot say which component stopped paying, or where along the model-capability axis it happened; that needs the harness arm run across earlier generations and the component ablations of Section 1. All three operators are attention-family; we state no claim beyond that family. Results are a snapshot of current frontier models and will move. The DSA reference is an interim competition kernel pending re-measurement against DeepSeek’s official kernels, and the table reports latency alone, so accuracy enters it only as a pass or fail against the oracle-relative bound of Section 3.3 rather than as a measured error margin. Agent coverage is partial: KDA-0.5 is a pre-1.0 version covering DSA and GDN only, and neither it nor AKO reports FA4 results, so FA4 has no agent baseline at all. Our own coverage is partial too: the Opus 4.8 DSA mean rests on two shapes, the third having failed the correctness check. Agent-time budgets are reported but not equalized across systems. Finally, the naive-loop prompt was issued in Chinese (Appendix B); prompt language is an uncontrolled variable in our setup, and we did not test whether an English formulation elicits the same behaviour.

7 CONCLUSION

We asked whether an agent without the domain machinery around specialized kernel agents can still write the kernels. We answered it by comparing kernels end to end. Naive-loop kernels cross the expert reference line as the models improve. The weakest model in our sweep sits below that line on every operator. The strongest sits at or above it on all three (Figure 1). The same kernels then pass KDA-0.5 on both operators it covers, by geometric mean over shapes, and win five of the six shapes where both systems run.

So a goal sentence is enough, with all three harness components left to the model. The help those components provide has been thinning as the models improve. Against every specialized agent we could measure it is already negative. The template space the specialized workflow searches on GDN

prefill could not have expressed the kernel that won; there the harness bounded what was reachable. We cannot yet put a number on the aggregate cost. That takes the same model measured inside and outside a specialized harness, plus the component ablations we defer to the full study. What the data settles is the question we opened with. On these operators, specialized machinery is no longer what a competitive kernel requires. The systems that still carry it there have to show it pays. That claim ends where the naive loop’s reach ends. Past it, loop engineering still holds the work.

What we would hand over next is decision rights. The workflow above the model fixes its architecture. The assembler below it fixes its instruction schedule. Both decide using information the model has. The run in Section 5 reaches across both ends at once: it measures an instruction’s ceiling, then rewrites the architecture around the answer. We measured neither end ourselves. This is a direction for the next study.

REFERENCES

- AgentKernelArena authors. Agentkernelarena: Generalization-aware benchmarking of GPU kernel optimization agents, 2026. arXiv:2605.16819.
- AKO Team. AKO: Agentic kernel optimization. <https://github.com/TongmingLAIC/AKO>, 2026a.
- AKO Team. AKO4ALL: Agentic kernel optimization for all. <https://github.com/TongmingLAIC/AKO4ALL>, 2026b.
- AKO Team. AKO4X: Agentic kernel optimization, advanced and extensible. <https://github.com/TongmingLAIC/AKO4X>, 2026c.
- Anthropic. Claude code. <https://claude.com/claude-code>, 2025.
- Shiyi Cao, Ziming Mao, Joseph E. Gonzalez, and Ion Stoica. K-Search: LLM kernel generation via co-evolving intrinsic world model, 2026. arXiv:2602.19128.
- Tianqi Chen, Lianmin Zheng, Eddie Yan, Ziheng Jiang, Thierry Moreau, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems*, 2018.
- Cognition. Kevin: Multi-turn RL for generating CUDA kernels, 2025. arXiv:2507.11948.
- Weinan Dai, Hanlin Wu, Qiyang Yu, Huan-ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiaze Chen, Wei-Ying Ma, Ya-Qin Zhang, Jingjing Liu, Mingxuan Wang, Xin Liu, and Hao Zhou. CUDA agent: Large-scale agentic RL for high-performance CUDA kernel generation, 2026. arXiv:2602.24286.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.
- Tri Dao et al. Flashattention-4: Algorithm and kernel pipelining co-design for asymmetric hardware scaling. In *Proceedings of Machine Learning and Systems (MLSys)*, 2026. arXiv:2603.05451.
- DeepSeek-AI. FlashMLA: Efficient multi-head latent attention kernels. <https://github.com/deepseek-ai/FlashMLA>, 2025a.
- DeepSeek-AI. Deepseek-v3.2: Pushing the frontier of open large language models, 2025b. arXiv:2512.02556.
- Weihua Du, Jingming Zhuo, Yixin Dong, Andre Wang He, Weiwei Sun, Zeyu Zheng, Manupa Karunaratne, Ivan Fox, Tim Dettmers, Tianqi Chen, Yiming Yang, and Sean Welleck. AdaExplore: Failure-driven adaptation and diversity-preserving search for efficient kernel generation, 2026. arXiv:2604.16625.
- FlagOS-AI. Awesome LLM-driven kernel generation. <https://github.com/flagos-ai/awesome-LLM-driven-kernel-generation>, 2026. Continuously updated index of systems and their release status.

- FlashInfer contributors. [CAKE] long-term CAKE-generated kernel progress tracker. <https://github.com/flashinfer-ai/flashinfer/issues/4254>, 2026. FlashInfer issue #4254, opened 2026-07-29; lists the merged pull requests carrying CAKE’s recurrent KDA kernels, e.g. #4262 (B200 prefill) and #4279 (B200 decode).
- HAN Lab Kernel Mafia. Developing GPU kernels with agentic loops. <https://github.com/mit-han-lab/mlsys2026-flashinfer-contest>, 2026a. Technical report, with the per-operator phase prompts used for the competition runs.
- HAN Lab Kernel Mafia. MLSys26 cute-dsl kernels via KDA-v0.5. <https://github.com/mit-han-lab/mlsys2026-flashinfer-contest-solution>, 2026b. Released kernels, commit 7aad0429.
- Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter, 2024. arXiv:2407.01502.
- MIT HAN Lab. Kernel design agents: An agent-centric workflow for high-performance CUDA kernels. <https://github.com/mit-han-lab/kernel-design-agents>, 2026.
- NVIDIA. AVO: Agentic variation operators for autonomous evolutionary search, 2026. arXiv:2603.24517.
- OpenAI. Codex. <https://openai.com/codex>, 2025.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernelbench: Can LLMs write efficient GPU kernels?, 2025. arXiv:2502.10517.
- Qwen Team. Qwen3-next. <https://qwen.ai/blog?id=4074cca80393150c248e508aa62983f9cb7d27cd>, 2025.
- Sakana AI. The AI CUDA Engineer: Agentic CUDA kernel discovery, optimization and composition. <https://sakana.ai/ai-cuda-engineer/>, 2025.
- Eric Schkufza, Rahul Sharma, and Alex Aiken. Stochastic superoptimization. In *Proceedings of the 18th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2013.
- Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems*, 2024.
- Richard S. Sutton. The bitter lesson. <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>, 2019.
- Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, 2019.
- Jianghui Wang, Vinay Joshi, Saptarshi Majumder, Xu Chao, Bin Ding, Ziqiong Liu, Pratik Prabhanjan Brahma, Dong Li, Zicheng Liu, and Emad Barsoum. GEAK: Introducing Triton kernel AI agent and evaluation benchmarks, 2025. arXiv:2507.23194.
- Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, et al. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts, 2024. arXiv:2411.15114.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents, 2024. arXiv:2407.01489.
- Songlin Yang and Yu Zhang. FLA: A Triton-based library for hardware-efficient implementations of linear attention mechanism. <https://github.com/fla-org/flash-linear-attention>, 2024.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *International Conference on Learning Representations*, 2025.

Table 5: Measured latency in μs for every cell in Table 3, including the expert reference each speedup is taken against. All numbers are our own measurements on the node of Section 3.2. Dashes carry the same meaning as in Table 3.

Operator	Shape	Reference	Opus 4.8	Opus 5	Fable 5.1	KDA-0.5	AKO4X
FA4 bwd	MHA	1387	6407	1374	1465	–	–
	GQA	1459	3618	1313	1176	–	–
	varlen MHA	1639	3062	1832	1521	–	–
DSA fwd	heavy load	13	<i>fail</i>	8.122	11	10.3	11.4
	small batch	5	3.167	2.826	3.2	3.4	7.2
	mixed lengths	11.07	19.75	5.117	5.189	6.8	8.2
GDN fwd	long sequence	145.6	155.6	180	94.5	125.5	221
	single short seq.	7	9.16	4.288	3.862	4.9	17.6
	few long seqs.	133	233.8	133.1	95	105	193
MLA decode	single mid-length	15.98	–	–	6.48	–	–
	mid batch, mixed	31.25	–	–	13.17	–	–
	large batch	77.5	–	–	46.71	–	–
MLA prefill	short prefill	22.38	–	3.936	–	–	–
	mid prefill	26.46	–	13.02	–	–	–
	mixed large prefill	709.2	–	496.2	–	–	–
GQA decode	single request	12.07	–	7.92	–	–	–
	mid batch, short seqs.	7.26	–	–	3.96	–	–
	large batch	74.8	–	49.5	–	–	–

Zihao Ye, Lequn Chen, Ruihang Lai, et al. FlashInfer: Efficient and customizable attention engine for LLM inference serving, 2025. arXiv:2501.01005.

Zihao Ye, Yingyi Huang, Hongyi Jin, et al. CAKE: Compiler-agent co-design for frontier kernel evolution, 2026. arXiv:2608.12629.

Yang Yu, Peiyu Zang, Chi Hsu Tsai, Haiming Wu, Yixin Shen, Jialing Zhang, Haoyu Wang, Zhiyou Xiao, Jingze Shi, Yuyu Luo, Wentao Zhang, Chunlei Men, Guang Liu, and Yonghua Lin. Towards automated kernel generation in the era of LLMs, 2026. arXiv:2601.15727.

Zijian Zhang, Rong Wang, Shiyang Li, Yuebo Luo, Mingyi Hong, and Caiwen Ding. CudaForge: An agent framework with hardware feedback for CUDA kernel optimization, 2025. arXiv:2511.01884.

Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. Ansor: Generating high-performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.

A SHAPE CONFIGURATIONS

Each run targets one configuration below. Workload identifiers refer to the MLSys-2026 competition harness. Where a KDA-0.5 reference is given it is our own measurement of its published CuTe-DSL kernel at contest-solution 7aad0429, not the figure KDA reports.

FlashAttention-4 backward. The reference is the flash-attn-4 4.0.0b19 backward entry point bundled with the model-align/sglang:v0.5.19 image.

- *MHA*: $B = 8$, $N = 4096$, $H = 16$, $D = 128$, bf16, causal, layout (B, N, H, D) .
- *GQA*: as MHA, with $H_{KV} = 2$ (8:1).
- *varlen MHA*: 13 documents packed into $T = 32768$ tokens (lengths 4096, 217, 8192, 1500, 513, 2800, 6144, 700, 1777, 257, 3500, 1024, 2048; longest 8192), $H = 16$, $D = 128$, bf16, causal within each document, layout (T, H, D) with `cu_seqlens` marking the boundaries.

DeepSeek Sparse Attention forward. All three shapes share $H = 16$, a 512-wide compressed KV and 64-wide positional component, $\text{top-}k = 2048$, page size 64, an 8462-page pool, and bf16. Layouts are `q_nope` ($T, 16, 512$), `q_pe` ($T, 16, 64$), `ckv_cache` (8462, 64, 512), `kpe_cache` (8462, 64, 64), and `sparse_indices` ($T, 2048$) int32. What separates them is how many KV rows the requests actually touch.

- *heavy load*: $T = 8$ decode requests touching 288, 4, 1884, 21, 136, 2048, 42, and 335 rows (4758 in total). Workload 564007ac; KDA-0.5 reference 10.3 μs .
- *small batch*: $T = 2$, touching 18 and 11 rows (29 total). Workload 9d4a5f21; KDA-0.5 reference 3.4 μs .
- *mixed lengths*: $T = 2$, touching 6 and 337 rows (343 total). Workload 05f6de65; KDA-0.5 reference 6.8 μs .

Gated DeltaNet prefill forward. All three shapes use 4 query and key heads, 8 value heads, head dimension 128, bf16 for `q/k/v/a/b`, and fp32 for the recurrent state (`k-last`). Layouts are `q/k` ($T, 4, 128$), `v` ($T, 8, 128$), `a/b` ($T, 8$), `A_log/dt_bias` (8), `state` ($N, 8, 128, 128$) and `cu_seqlens` ($N+1$) int64, producing output ($T, 8, 128$) and `new_state` ($N, 8, 128, 128$).

- *long sequence*: $T = 8192$ tokens packed into $N = 20$ sequences (753, 371, 161, 41, 105, 66, 23, 31, 19, 15, 2433, 39, 22, 3371, 20, 42, 219, 15, 45, 401). Workload a9540651; KDA-0.5 takes its `gdn_core` path, 125.5 μs .
- *single short seq.*: $T = 30$ tokens, $N = 1$. Workload 62447caf; KDA-0.5 takes its `gdn_small` path, 4.9 μs .
- *few long seqs.*: $T = 5709$ tokens packed into $N = 2$ sequences (5637 and 72). Workload 07aa7922; KDA-0.5 takes a chunked path with three launches per call, 105 μs .

B PROMPTS AND AGENT CONFIGURATIONS

B.1 THE NAIVE-LOOP PROMPT

Below is the complete instruction given to the naive-loop agent; nothing else about the task is supplied in context. This is the default prompt for every naive-loop run in this paper unless stated otherwise. The runs were conducted in Chinese, so we give the original verbatim, followed by a literal English translation. The translation is provided for readability and was not itself run; we have not verified that the two formulations elicit identical behaviour (Section 6).

Original (as run).

这是我们要的目标算子（算子 config），请先在 B200 上测量它的 device time 时间。从 0 开始，无任何限制尽可能超过他的速度。请务必确保正确性。最好通过先写一个 fp32 的 naive 慢实现，再和 ref 比他们两的差这样可以获得一个下限容差范围再乘个值（e.g. 1），然后用这个容差来对比你的实现和 fp32 慢实现的。

Literat English translation.

This is the target operator we want (operator config). First measure its device time on a B200. Starting from zero, with no restrictions whatsoever, beat that speed by as much as you can. Make sure it is correct. The best way is to first write a naive, slow fp32 implementation, then compare it against the reference; the difference between the two gives a lower bound for the tolerance, which you multiply by a factor (e.g. 1), and then use that tolerance to compare your implementation against the slow fp32 implementation.

Three things about this prompt carry the design of Section 3.1. “Starting from zero” means the agent is given no skeleton, reference implementation, or partial kernel to edit; it writes the operator from nothing. “With no restrictions whatsoever” is the absence of the workflow component made

explicit to the model: no algorithm is mandated and no order of optimization steps is prescribed. The operator config referenced in the first sentence is the single shape configuration that run targets (Section 3.2). The last two sentences are the whole of the evaluation component: instead of calling a gate the harness supplies, the agent is told to construct one, with its own naive fp32 implementation as the oracle and the reference’s own distance from that oracle, times the factor, as the bound its kernel must meet.

B.2 HARNESS AND AGENT CONFIGURATION

Every naive-loop run is a Claude Code session started with the `/goal` command, with the prompt above as its objective. All naive-loop runs used the same client-side configuration across the three model generations. The reasoning effort level was set to `high` for every run (Opus 4.8, Opus 5, and Fable 5.1), so effort is held fixed and the model generation is the only variable along that axis. Opus 5 and Fable 5.1 were run as their 1M-token context variants (`claude-opus-5[1m]`, `claude-fable-5-1[1m]`); Opus 4.8 was run with its standard context window. [TODO: Claude Code version; tool access; evaluation-loop invocation details]